```
long wait_event_timeout(wait_queue_head_t q, condition, long
timeout);
```

Small delays: Sometimes, kernel code, like a real driver, needs to calculate very short delays in order to synchronise with hardware. Therefore, one cannot use *jiffies* for the delay. The kernel functions *udelay* and *mdelay* are used for short waits. Their prototypes are:

```
#include <linux/delay.h>
void udelay(unsigned long usecs);
void mdelay(unsigned long msecs);
```

The *udelay* function delays execution by busy looping for the specified number of *microseconds*. The *mdelay* function delays execution for the specified number of *milliseconds*. 1 second = 1,000 milliseconds = 1,000,000 microseconds, and *udelay (150)* is a delay for 150 µs. The *udelay()* function is implemented as a loop that knows how many iterations can be executed in a given period of time. The *mdelay()* function is then implemented in terms of *udelay()*. Because the kernel knows how many loops the processor can complete in a second, the *udelay()* function simply scales that value to the correct number of loop iterations for the given delay.

Task scheduling and the use of the kernel timer

In multi-tasking OSs, many tasks run at the same time. Providing the appropriate resource to the task is called *task scheduling*. The tool that distributes the available resources to the tasks is called a *task scheduler*. This is also called the *process scheduler,* and is the part of the kernel that decides which task to run next. It is one of the essential factors of a multi-tasking OS. One feature many drivers need is the ability to schedule the execution of some tasks at a later time without resorting to interrupts. Linux offers three different interfaces for this purpose: task *queues, tasklets*, and *kernel timers*.

Task *queues* and *tasklets* provide a flexible utility for scheduling execution at a later time, and are triggered by the kernel itself. They are only used to manage hardware that cannot generate interrupts. They always run in the interrupt time. And run only once, though scheduled to run multiple times. Kernel timers are used to schedule a task to run at a specific time in the future. Moreover, they are easy to use, and also do not re-register themselves, unlike task queues that do.

At times, one needs to execute operations detached from any process context, like finishing a lengthy shutdown operation. In that case, delaying the return from the *close()* (the function for closing the file descriptor; it returns 0 or 1 for success, r for failure, etc) wouldn't be fair to the application program. Using a task *queue* would be wasteful, because a queued task must continually re-register itself until the requisite time has passed.

The kernel timers are organised as a double linked list. *add_timer* and *del_timer* are the functions used to add and remove a timer from the list. Once the timer expires it is automatically removed from the list. Bidirectional in double linked list is an advantage since in immediate previous timer is not necessitated for deletion. A timer is marked by its timeout value (in *jiffies*) and the function that is to be called on the time value expiry. The timer handler receives an argument, which is stored in the data structure, together with a pointer to the handler itself. The data structure of the timer is in <linux/timer.h>:

```
struct timer_list {
 struct timer_list *next; /*hold the address next node*/
 struct timer_list *prev; /*hold the previous next node*/
 unsigned long expires; /* the timeout, in jiffies */
 unsigned long data; /* argument to the handler */
 void (*function)(unsigned long); /* handler of the timeout */
 volatile int running;
};
```

Here, *expires* denotes *jiffies*. The execution of *timer->function* will last till *jiffies* is equal to or greater than *timer->expires*. The timeout is an absolute value equal to the current value of *jiffies* and the amount of the desired delay. The first step in creating a timer is to initialise it:

```
Struct timer_list K_timer ;
```

Now it should be initialised.

```
Init_timer(&K_timer);
```

The *timer_list* structure is initialised once; the function *add_timer* inserts it into a sorted list, which is then polled a 100 times per second, more or less. The *add_timer* is declared in *<time.h>* and defined in *<timer.c>*

```
Stereotype: Extern void add_timer( struct timer_list *timer )
```

Now the timer should be activated.

```
Add_timer(&k_timer);
```

Even systems (such as the Alpha) that run with a higher clock interrupt frequency do not check the timer list more often than that—the added timer resolution would not justify the cost of the extra passes through the list. END

References

'Linux Kernel Development' by Robert Love

By: Debasree Panda

The author is an open source developer. His area of expertise is teradata databases.